

Outline

- Multi-processors
- Shared-memory architectures
- Memory synchronization

Increasing Importance of Multiprocessing

- ❑ Inability to exploit more ILP
- ❑ Power & silicon costs grow faster than performance
- ❑ Growing interest in
 - High-end servers as cloud computing & software as a service
 - Data-intensive applications
- ❑ Lower interest in increasing desktop performance
- ❑ Better understanding of how to use multiprocessors effectively
- ❑ Replication of a core is relatively easy than a completely new design

Vectors, MMX, GPUs vs. Multiprocessors

- Vectors, MMX, GPUs
 - SIMD
- Multiprocessors
 - MIMD

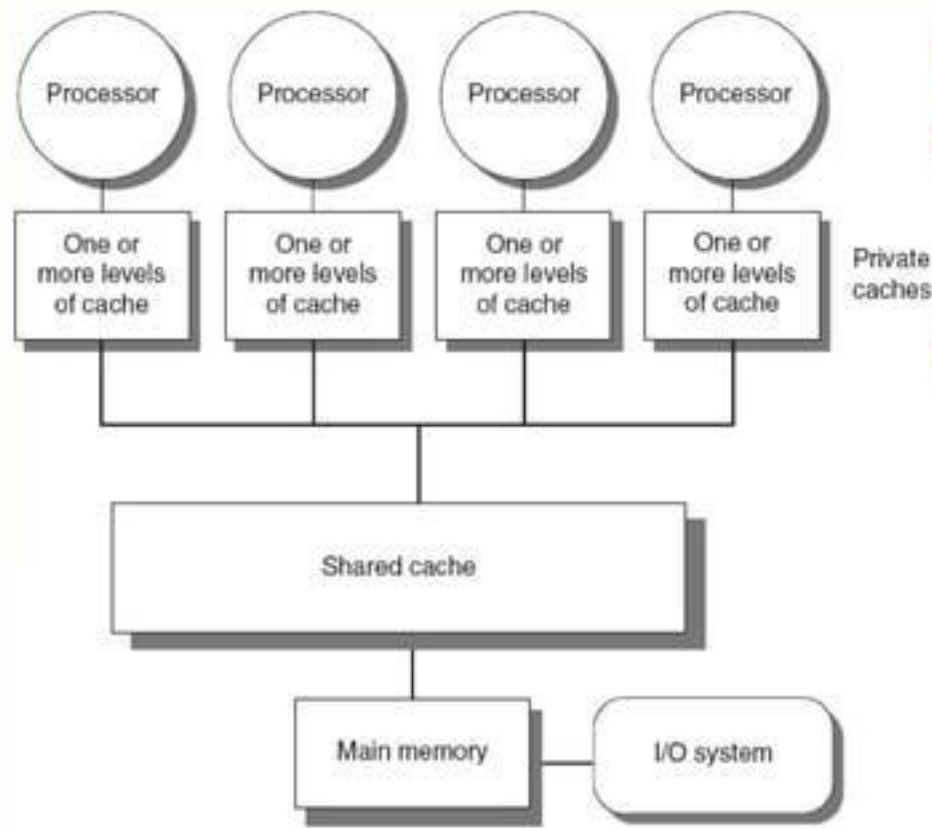
Multiprocessor Architecture

- MIMD multiprocessor with n processors
 - Need n threads or processors to keep it fully utilized
- Thread-Level parallelism
 - Uses MIMD model
 - Have multiple program counters
 - Targeted for tightly-coupled shared-memory multiprocessors
 - Communication among threads through shared memory

Definition – Grain Size

- Amount of computation assigned to each thread
- Threads can be used for data-level parallelism, but overheads may outweigh benefits

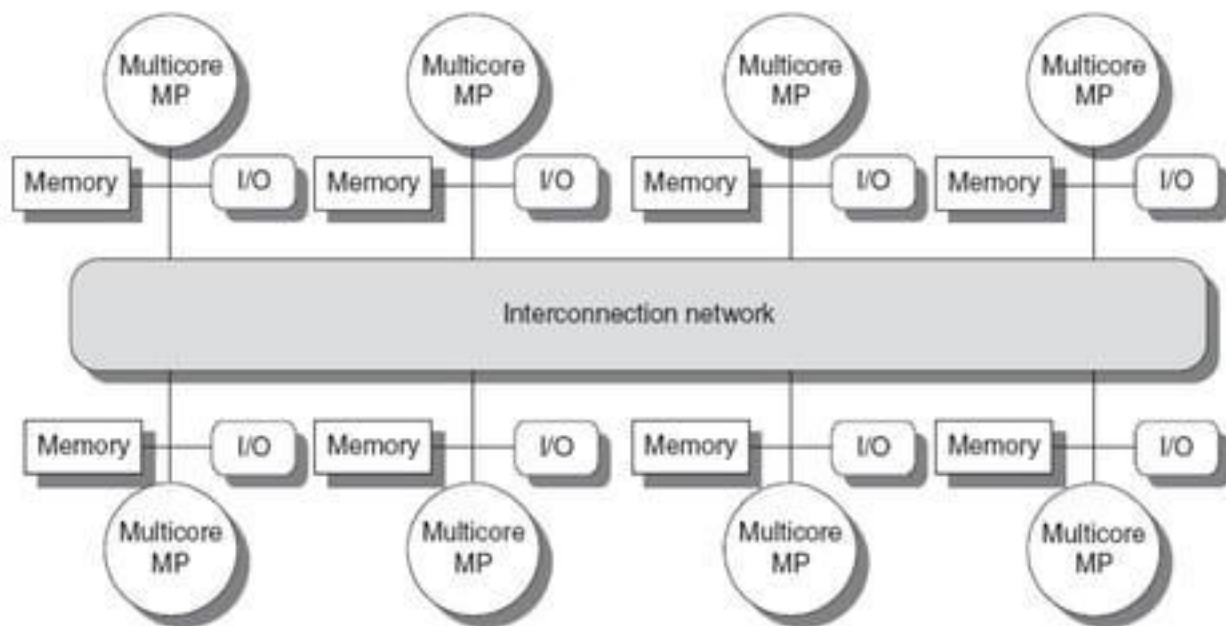
Symmetric Multiprocessors (SMP)



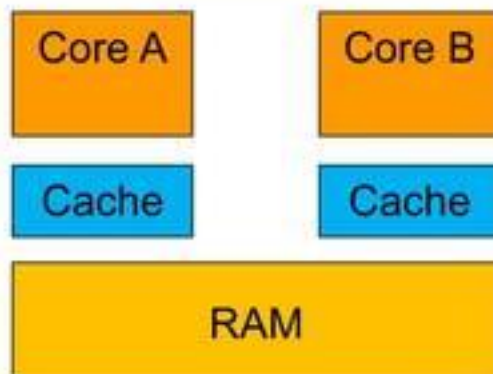
- Small no of cores
- Share single memory with uniform memory latency
- A.k.a. Uniform Memory Access (UMA)

Distributed Shared Memory (DSM)

- ❑ Memory distributed among processors
- ❑ Processors connected via direct (switched) & non-direct (multi-hop) interconnection networks
- ❑ A.k.a. Non-Uniform Memory Access/latency (NUMA)



Cache Coherence



Time	Event	Cache contents for processor A	Cache contents for processor B	Memory contents for location X
0				1
1	Processor A reads X	1		1
2	Processor B reads X	1	1	1
3	Processor A stores 0 into X	0	1	0

Cache Coherence (Cont.)

- ❑ Processors may see different values through their caches
 - Caching shared data introduces new problems
- ❑ In a coherent memory system read of a data item returns the most recently written value
- ❑ 2 aspects
 - Coherence
 - Consistency

1. Coherence

- Defines behavior of reads & writes to the same memory location
- What value can be returned by a read
 - All reads by any processor must return most recently written value
 - Writes to same location by any 2 processors are seen in same order by all processors
 - Serialized writes to same location

2. Consistency

- Defines behavior of reads & writes with respect to access to other memory locations
- When a written value will be returned by a read
- If a processor writes location x followed by location y , any processor that sees new value of y must also see new value of x
 - Serialized writes different locations

Enforcing Coherence

- Coherent caches provide
 - Migration – movement of data
 - Replication – multiple copies of data

- Cache coherence protocols
 - Snooping
 - Each core tracks sharing status of each of its blocks
 - Distributed
 - Directory based
 - Sharing status of each block kept in a single location
 - Centralized

Snooping Coherence Protocol

□ Write invalidate protocol

- On write, invalidate all other cached copies
- Use bus itself to serialize
 - Write can't complete until bus access is obtained
 - Concurrent writes?
 - One who obtains bus wins
- Most common implementation

Processor activity	Bus activity	Contents of processor A's cache	Contents of processor B's cache	Contents of memory location X
				0
Processor A reads X	Cache miss for X	0		0
Processor B reads X	Cache miss for X	0	0	0
Processor A writes a 1 to X	Invalidation for X	1		0
Processor B reads X	Cache miss for X	1	1	1

Snooping Coherence Protocol (Cont.)

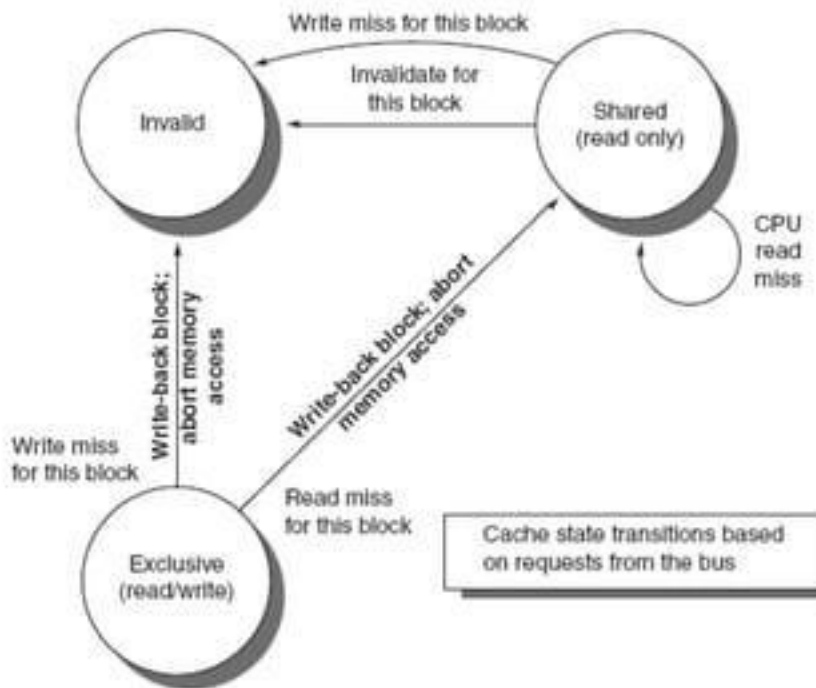
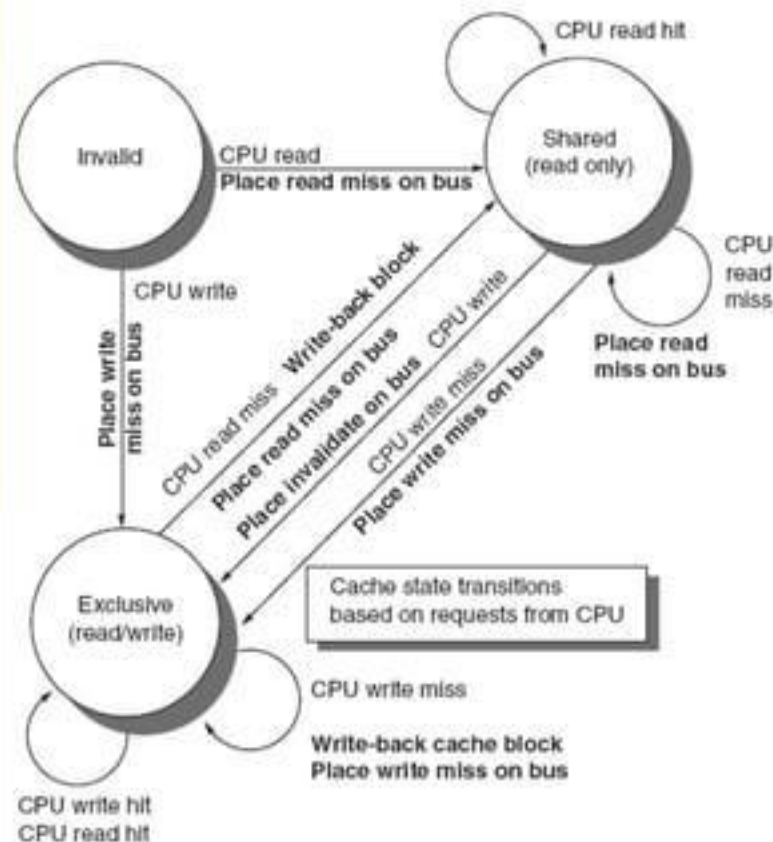
□ Write update protocol

- A.k.a. Write broadcast protocol
- On write, update all copies
- Need more bandwidth

Snooping Coherence Protocol – Implementation Techniques

- Locating an item when a read miss occurs
 - Write-through cache
 - Recent copy in memory
 - Write-back cache
 - Every processor snoops every address placed on shared bus
 - If a processor finds it has a dirty block, updated block is sent to requesting processor
- Cache lines marked as shared or exclusive/modified
 - Only writes to shared lines need an invalidate broadcast
 - After this, line is marked as exclusive

Snoopy Coherence Protocol – State Transition Example



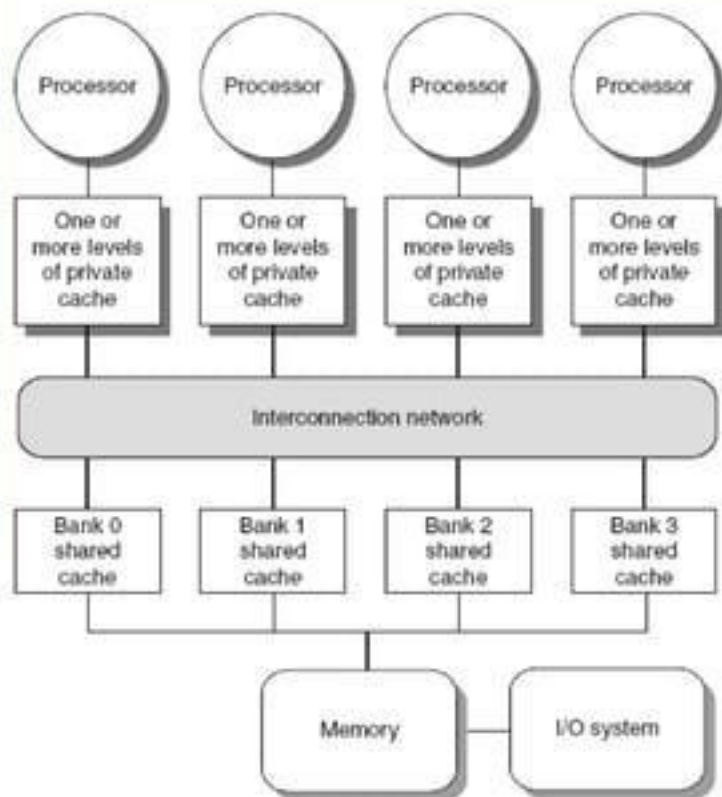
Snoopy Coherence Protocol – State Transition Example

Request	Source	State of addressed cache block	Type of cache action	Function and explanation
Read hit	Processor	Shared or modified	Normal hit	Read data in local cache.
Read miss	Processor	Invalid	Normal miss	Place read miss on bus.
Read miss	Processor	Shared	Replacement	Address conflict miss: place read miss on bus.
Read miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place read miss on bus.
Write hit	Processor	Modified	Normal hit	Write data in local cache.
Write hit	Processor	Shared	Coherence	Place invalidate on bus. These operations are often called upgrade or <i>ownership</i> misses, since they do not fetch the data but only change the state.
Write miss	Processor	Invalid	Normal miss	Place write miss on bus.
Write miss	Processor	Shared	Replacement	Address conflict miss: place write miss on bus.
Write miss	Processor	Modified	Replacement	Address conflict miss: write-back block, then place write miss on bus.
Read miss	Bus	Shared	No action	Allow shared cache or memory to service read miss.
Read miss	Bus	Modified	Coherence	Attempt to share data: place cache block on bus and change state to shared.
Invalidate	Bus	Shared	Coherence	Attempt to write shared block; invalidate the block.
Write miss	Bus	Shared	Coherence	Attempt to write shared block; invalidate the cache block.
Write miss	Bus	Modified	Coherence	Attempt to write block that is exclusive elsewhere; write-back the cache block and make its state invalid in the local cache.

Snoopy Coherence Protocol – Issues

- ❑ Operations are not atomic
 - e.g., detect miss, acquire bus, receive a response
 - Applies to both reads & writes
- ❑ Creates possibility of deadlock & races
- ❑ Actual Snoopy Coherence Protocols are more complicated

Coherence Protocols – Extensions

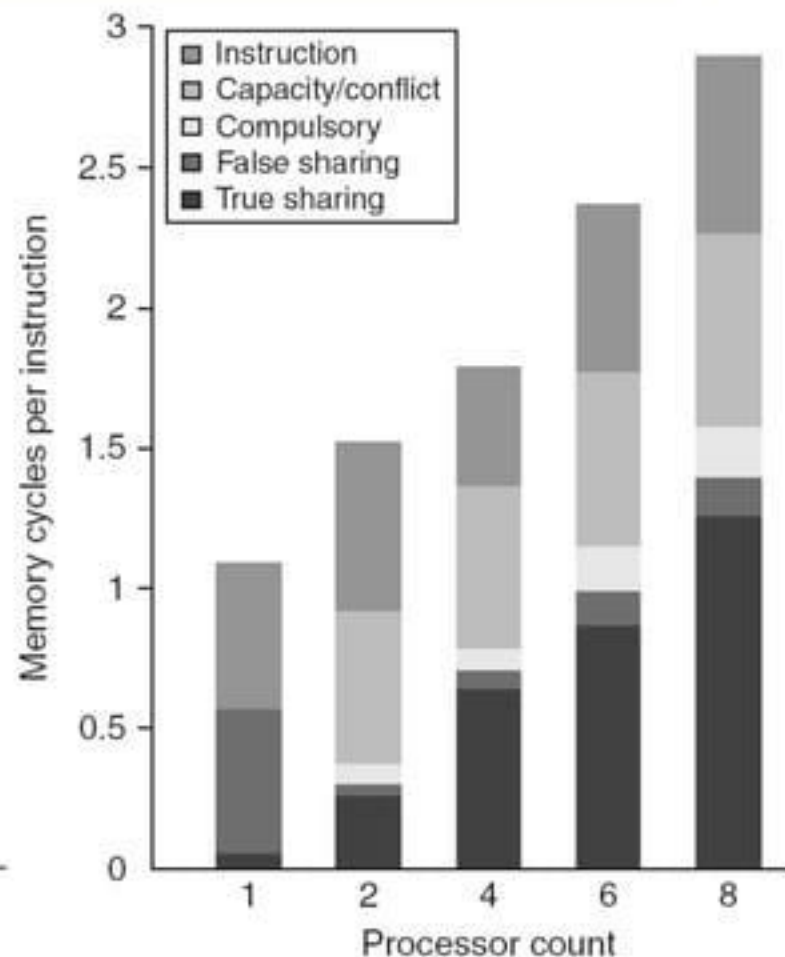
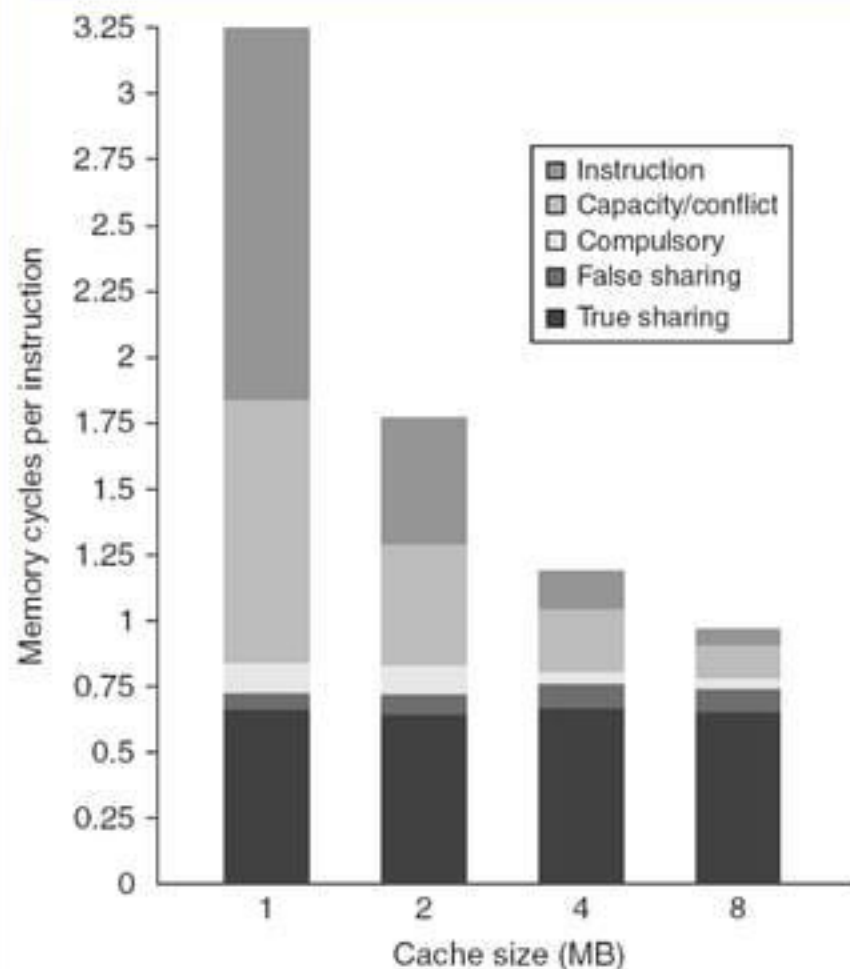


- Shared memory bus & snooping bandwidth is bottleneck for scaling symmetric multiprocessors
 - Duplicating tags
 - Place directory in outermost cache
 - Use crossbars or point-to-point networks with banked memory

Cache Coherence – Performance

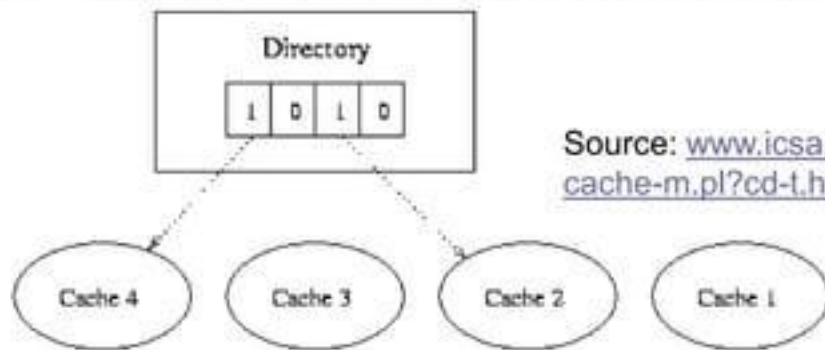
- Coherence influences cache miss rate
 - Coherence misses
 - True sharing misses
 - Write to shared block (transmission of invalidation)
 - Read an invalidated block
 - False sharing misses
 - Read an unmodified word in an invalidated block

Performance Study – Commercial Workload



Directory-Based Cache Coherence

- Sharing status of each physical memory block kept in a single location
- Approaches
 - Central directory for memory or common cache
 - For Symmetric Multiprocessors (SMPs)
 - Distributed directory
 - For Distributed Shared Memory (DSM) systems
 - Overcomes single point of contention in SMPs

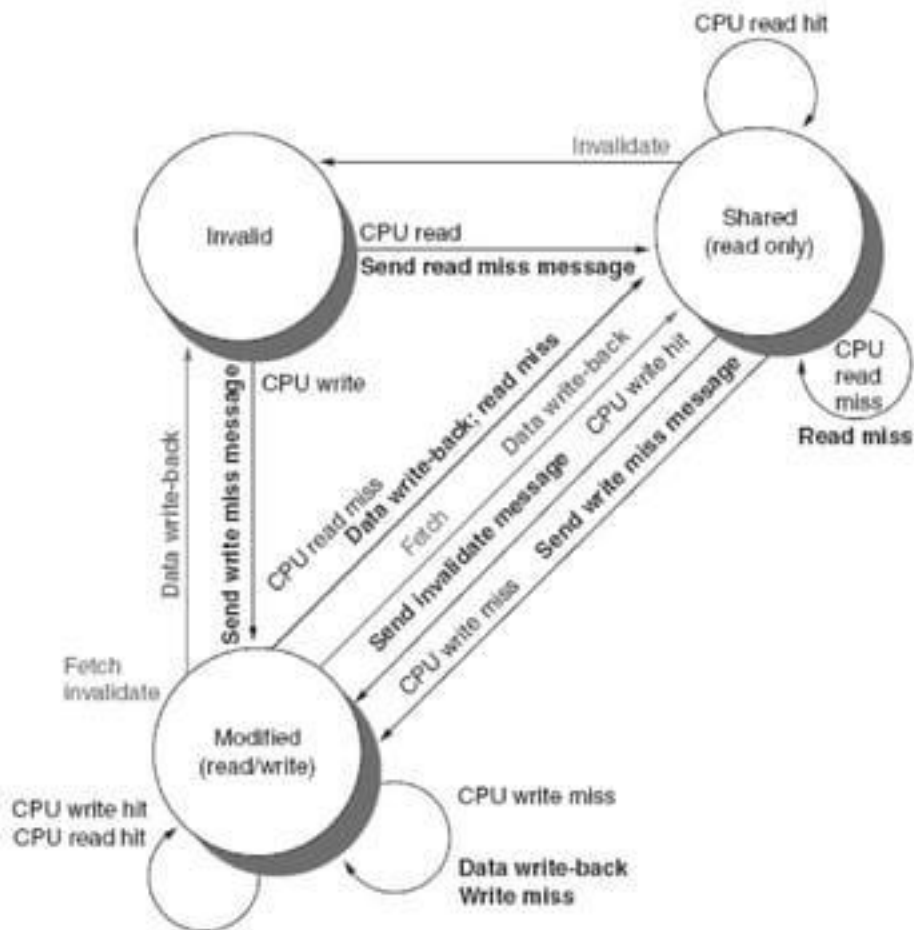


Source: www.icsa.inf.ed.ac.uk/cgi-bin/hase/dir-cache-m.pl?cd-t.html,cd-f.html,menu1.html

Directory Protocols

- For each block, maintain state
 - Shared
 - 1 or more nodes have the block cached, value in memory is up-to-date
 - Set of node IDs
 - Uncached
 - Modified
 - Exactly 1 node has a copy of the cache block, value in memory is out-of-date
 - Owner node ID
- Directory maintains block states & sends invalidation messages

Directory Protocols (Cont.)



Directory Protocols (Cont.)

□ For uncached block

■ Read miss

- Requesting node is sent requested data & is made the only sharing node, block is **now shared**

■ Write miss

- Requesting node is sent requested data & becomes the sharing node, block is **now exclusive (modified)**

□ For shared block

■ Read miss

- Requesting node is sent requested data from memory, node is added to sharing set

■ Write miss

- Requesting node is sent value, all nodes in sharing set are sent invalidate messages, sharing set only contains requesting node, block is **now exclusive**

Directory Protocols (Cont.)

□ For exclusive block

■ Read miss

- Owner is sent a data fetch message, block becomes shared, owner sends data to directory, data written back to memory, sharers set contains old owner & requestor

■ Data write back

- Block becomes uncached, sharer set is empty

■ Write miss

- Message is sent to old owner to invalidate & send value to directory, requestor becomes new owner, block **remains exclusive**

Directory Protocols (Cont.)

Message type	Source	Destination	Message contents	Function of this message
Read miss	Local cache	Home directory	P, A	Node P has a read miss at address A; request data and make P a read sharer.
Write miss	Local cache	Home directory	P, A	Node P has a write miss at address A; request data and make P the exclusive owner.
Invalidate	Local cache	Home directory	A	Request to send invalidates to all remote caches that are caching the block at address A.
Invalidate	Home directory	Remote cache	A	Invalidate a shared copy of data at address A.
Fetch	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; change the state of A in the remote cache to shared.
Fetch/invalidate	Home directory	Remote cache	A	Fetch the block at address A and send it to its home directory; invalidate the block in the cache.
Data value reply	Home directory	Local cache	D	Return a data value from the home memory.
Data write-back	Remote cache	Home directory	A, D	Write-back a data value for address A.

Synchronization Operations

□ Basic building blocks

- Atomic exchange
 - Swaps register with memory location
- Test-and-set
 - Sets under condition
- Fetch-and-increment
 - Reads original value from memory & increments it in memory
- Requires memory read & write in uninterruptable instruction
- load linked/store conditional
 - If contents of memory location specified by load linked are changed before store conditional to same address, store conditional fails

Implementing Locks Using Coherence

□ Spin lock

□ If no coherence

	DADDUI R2,R0,#1	
lockit:	EXCH R2,0(R1)	;atomic exchange
	BNEZ R2,lockit	;already locked?

□ If coherence

lockit:	LD R2,0(R1)	;load of lock
	BNEZ R2,lockit	;not available-spin
	DADDUI R2,R0,#1	;load locked value
	EXCH R2,0(R1);	swap
	BNEZ R2,lockit;	branch if lock wasn't 0

Implementing Locks – Advantages

■ Reduces memory traffic

- During each iteration, current lock value can be read from cache
- Locality of accessing lock

Step	P0	P1	P2	Coherence state of lock at end of step	Bus/directory activity
1	Has lock	Begins spin, testing if lock = 0	Begins spin, testing if lock = 0	Shared	Cache misses for P1 and P2 satisfied in either order. Lock state becomes shared.
2	Set lock to 0	(Invalidate received)	(Invalidate received)	Exclusive (P0)	Write invalidate of lock variable from P0.
3		Cache miss	Cache miss	Shared	Bus/directory services P2 cache miss; write-back from P0; state shared.
4		(Waits while bus/directory busy)	Lock = 0 test succeeds	Shared	Cache miss for P2 satisfied
5		Lock = 0	Executes swap, gets cache miss	Shared	Cache miss for P1 satisfied
6		Executes swap, gets cache miss	Completes swap: returns 0 and sets lock = 1	Exclusive (P2)	Bus/directory services P2 cache miss; generates invalidate; lock is exclusive.
7		Swap completes and returns 1, and sets lock = 1	Enter critical section	Exclusive (P1)	Bus/directory services P1 cache miss; sends invalidate and generates write-back from P2.
8		Spins, testing if lock = 0			None

Step	P0	P1	P2	Coherence state of lock at end of step	Bus/directory activity
1	Has lock	Begins spin, testing if lock = 0	Begins spin, testing if lock = 0	Shared	Cache misses for P1 and P2 satisfied in either order. Lock state becomes shared.
2	Set lock to 0	(Invalidate received)	(Invalidate received)	Exclusive (P0)	Write invalidate of lock variable from P0.
3		Cache miss	Cache miss	Shared	Bus/directory services P2 cache miss; write-back from P0; state shared.
4		(Waits while bus/directory busy)	Lock = 0 test succeeds	Shared	Cache miss for P2 satisfied
5		Lock = 0	Executes swap, gets cache miss	Shared	Cache miss for P1 satisfied
6		Executes swap, gets cache miss	Completes swap: returns 0 and sets lock = 1	Exclusive (P2)	Bus/directory services P2 cache miss; generates invalidate; lock is exclusive.
7		Swap completes and returns 1, and sets lock = 1	Enter critical section	Exclusive (P1)	Bus/directory services P1 cache miss; sends invalidate and generates write-back from P2.
8		Spins, testing if lock = 0			None

Summary

- Multi-processors
 - Create new problems, e.g., cache coherency
- Aspects of cache coherence
 - Coherence
 - Consistency
- Shared-memory architectures
 - Snooping
 - Directory based
- Memory synchronization